

18.2 Shooting to a Fitting Point

The shooting method described in §18.1 tacitly assumed that the “shots” would be able to traverse the entire domain of integration, even at the early stages of convergence to a correct solution. In some problems it can happen that, for very wrong starting conditions, an initial solution can’t even get from x_1 to x_2 without encountering some incalculable, or catastrophic, result. For example, the argument of a square root might go negative, causing the numerical code to crash. Simple shooting would be stymied.

A different, but related, case is where the endpoints are both singular points of the set of ODEs. One frequently needs to use special methods to integrate near the singular points, analytic asymptotic expansions, for example. In such cases it is feasible to integrate in the direction *away* from a singular point, using the special method to get through the first little bit and then reading off “initial” values for further numerical integration. However, it is generally not feasible to integrate *into* a singular point. Usually the desired boundary condition is that one wants a regular solution at the singular point, but integrating into a singularity is guaranteed to pick out a singular solution, which by definition is growing as one integrates inward. Any small numerical inaccuracy will include some admixture of the “wrong” solution, which grows and swamps the desired solution.

The solution to the above-mentioned difficulties is *shooting to a fitting point*. Instead of integrating from x_1 to x_2 , we integrate first from x_1 to some point x_f that is *between* x_1 and x_2 ; and second from x_2 (in the opposite direction) to x_f .

If (as before) the number of boundary conditions imposed at x_1 is n_1 , and the number imposed at x_2 is n_2 , then there are n_2 freely specifiable starting values at x_1 and n_1 freely specifiable starting values at x_2 . (If you are confused by this, go back to §18.1.) We can therefore define an n_2 -vector $\mathbf{V}^{(1)}$ of starting parameters at x_1 and a prescription `load1(x1,v1)` for mapping $\mathbf{V}^{(1)}$ into a $\mathbf{y}$ that satisfies the boundary conditions at x_1 :

$$y_i(x_1) = y_i(x_1; V_0^{(1)}, \dots, V_{n_2-1}^{(1)}) \quad i = 0, \dots, N-1 \quad (18.2.1)$$

Likewise we can define an n_1 -vector $\mathbf{V}^{(2)}$ of starting parameters at x_2 and a prescription `load2(x2,v2)` for mapping $\mathbf{V}^{(2)}$ into a $\mathbf{y}$ that satisfies the boundary conditions at x_2 :

$$y_i(x_2) = y_i(x_2; V_0^{(2)}, \dots, V_{n_1-1}^{(2)}) \quad i = 0, \dots, N-1 \quad (18.2.2)$$

We thus have a total of N freely adjustable parameters in the combination of $\mathbf{V}^{(1)}$ and $\mathbf{V}^{(2)}$. The N conditions that must be satisfied are that there be agreement in N components of $\mathbf{y}$ at x_f between the values obtained integrating from one side and from the other,

$$y_i(x_f; \mathbf{V}^{(1)}) = y_i(x_f; \mathbf{V}^{(2)}) \quad i = 0, \dots, N-1 \quad (18.2.3)$$

In some problems, the N matching conditions can be better described (physically, mathematically, or numerically) by using N different functions F_i , $i = 0 \dots N-1$, each possibly depending on the N components y_i . In those cases, (18.2.3) is replaced by

$$F_i[\mathbf{y}(x_f; \mathbf{V}^{(1)})] = F_i[\mathbf{y}(x_f; \mathbf{V}^{(2)})] \quad i = 0, \dots, N-1 \quad (18.2.4)$$

In the program below, a user-supplied function or functor, called `score(xf,y)` in the routine, is supposed to map an input N -vector y into an output N -vector F . In most cases, you can simply use the identity mapping $F = y$.

Shooting to a fitting point uses globally convergent Newton-Raphson exactly as in §18.1. Comparing closely with the routine `Shoot` of the previous section, you should have no difficulty in understanding the following routine `Shootf`. The main differences in use are that you have to supply both `load1` and `load2`. Also, in the calling program you must supply initial guesses for `v1[0..n2-1]` and `v2[0..n1-1]`. Once again, a sample program illustrating shooting to a fitting point is given in §18.4.

template <class L1, class L2, class R, class S> shootf.h

struct Shootf {

Functor for use with `newt` to solve a two-point boundary value problem by shooting to a fitting point.

<code>Int nvar,n2;</code>	<code>nvar</code> is the number of coupled ODEs.
<code>Doub x1,x2,xf;</code>	Start and end points and fitting point.
<code>L1 &load1;</code>	<code>load1</code> and <code>load2</code> supply initial values for the ODEs.
<code>L2 &load2;</code>	
<code>R &d;</code>	Supplies derivative information to the ODE integrator.
<code>S &score;</code>	Computes the mismatch of the solutions at the fitting point.
<code>Doub atol,rtol;</code>	
<code>Doub h1,hmin;</code>	
<code>VecDoub y,f1,f2;</code>	
<code>Shootf(Int nvarr, Int nn2, Doub xx1, Doub xx2, Doub xxf, L1 &load1,</code>	
<code>L2 &load2, R &dd, S &scoree) : nvar(nvarr), n2(nn2), x1(xx1),</code>	
<code>x2(xx2), xf(xxf), load1(load1), load2(load2), d(dd),</code>	
<code>score(scoree), atol(1.0e-14), rtol(atol), hmin(0.0), y(nvar),</code>	
<code>f1(nvar), f2(nvar) {}</code>	

Routine for use with `newt` to solve a two-point boundary value problem for `nvar` coupled ODEs by shooting from `x1` and `x2` to a fitting point `xf`. Initial values for the `nvar` ODEs at `x1` are generated from the `n2` coefficients `v1` and the user-supplied routine `load1`. Likewise, those at `x2` are from the `n1=nvar-n2` coefficients `v2`, using `load2`. The coefficients `v1` and `v2` should be stored in a single array `v[0..nvar-1]` in the main program with `v1` in `v[0..n2-1]` and `v2` in `v[n2..nvar-1]`.

`VecDoub operator() (VecDoub_I &v) {`

This is the functor used by `newt`. It integrates the ODEs to `xf` using an eighth-order Runge-Kutta method with absolute and relative tolerances `atol` and `rtol`, initial stepsize `h1`, and minimum stepsize `hmin`. At `xf` it calls the user-supplied routine `score` to evaluate the `nvar` functions `f1` and `f2` that ought to match at `xf`. The differences are returned on output. `newt` uses a globally convergent Newton's method to adjust the values of `v` until the differences are zero. A user-supplied function or functor `d` supplies derivative information to the ODE integrator (see Chapter 17).

```

    VecDoub v2(nvar-n2,&v[n2]);
    h1=(x2-x1)/100.0;
    y=load1(x1,v);
    Output out;
    Odeint<StepperDopr853<R> > integ1(y,x1,xf,atol,rtol,h1,hmin,out,d);
    integ1.integrate();
    f1=score(xf,y);
    y=load2(x2,v2);
    Odeint<StepperDopr853<R> > integ2(y,x2,xf,atol,rtol,h1,hmin,out,d);
    integ2.integrate();
    f2=score(xf,y);
    for (Int i=0;i<nvar;i++) f1[i] -= f2[i];
    return f1;
}
};
```

There are boundary value problems where even shooting to a fitting point fails